INTEGRATION GUIDE

메신저 링크 · 첨부파일 클릭 연동 가이드

답변 속 링크와 첨부파일을 눌렀을 때
닫기 버튼이 없거나, **뒤로가기가 메인으로 떨어지는** 문제를
링크 종류별 라우팅으로 해결하는 방법입니다.

대상 · 모바일 웹뷰 담당 개발자 (iOS / Android)

SDK · @sendbird/ai-agent-messenger-react

예제 코드 · `web/src/libs/linkHandler.ts`

1 현상과 원인

> 메신저 답변 속 링크·첨부파일을 누를 때 두 가지 문제가 보고되었습니다.
> ① **PDF를 누르면 닫기 버튼이 없는 화면**이 열려 돌아갈 수 없음
> ② **자사 서비스 링크를 누르면 히스토리 없이 이동**하여, 뒤로가기 시 메인홈으로 떨어짐

이미지와 그 외 링크는 처리 경로가 다릅니다

SDK는 **이미지·동영상만 내부 뷰어**로 표시합니다. 그 외는 모두 **새 창 열기를 요청**합니다.

```
이미지 · 동영상          →  SDK 내부 뷰어            →  닫기 버튼 있음 ✅

PDF · 기타 첨부파일      →  window.open(url, '_blank')  →  앱이 처리  →  ?
마크다운 링크            →  window.open(url, '_blank')  →  앱이 처리  →  ?
인용 · CTA 버튼          →  window.open(url, '_blank')  →  앱이 처리  →  ?
```

핵심은 `_blank` 입니다

`_blank`는 **"새 컨텍스트에서 열어라"**는 뜻이고, **새 컨텍스트에는 히스토리가 없습니다.** 그래서 뒤로가기를 눌러도 돌아갈 지점이 없어 앱이 메인 화면으로 떨어집니다.

또한 `window.open`은 **요청일 뿐**이고 실제로 어디에 어떻게 열지는 전적으로 앱이 정합니다. 웹뷰 환경에서는 다음 문제가 생깁니다.

환경	기본 동작과 문제
iOS (WKWebView)	앱이 `WKUIDelegate`로 처리하지 않으면 **무시**됩니다. 처리하더라도 닫기 UI 없는 새 `WKWebView`를 띄우면 **돌아갈 방법이 없습니다.**
Android (WebView)	같은 WebView에서 열면 **챗봇 화면을 덮어버립니다.** 뒤로가기 처리가 없으면 대화로 복귀할 수 없습니다.

> **참고 — SDK는 브라우저 히스토리를 사용하지 않습니다**
> 메신저 열기, 대화 ↔ 목록 전환 등 SDK 내부 화면 전환은 `pushState` / `popstate`를 전혀 사용하지 않습니다. 따라서 **복귀 지점 관리는 앱의 몫**입니다.

2 해결 방식 — 링크 종류별 라우팅

SDK가 제공하는 `handlers.onClickLink`로 **`window.open`이 호출되기 전에 가로채서** URL 종류에 따라 네이티브로 분기합니다.

```
[변경 전]
 모든 링크       →  window.open('_blank')  →  히스토리 없음 / 닫기 UI 없음 ❌

[변경 후]
 자사 서비스 링크 →  onClickLink  →  navigate_internal  →  앱 내비게이션 스택에 push  ✅
                                                          (뒤로가기로 대화 복귀)

 외부 링크 · PDF  →  onClickLink  →  open_external      →  OS 기본 외부 뷰어      ✅
                                                          (닫기 버튼 기본 제공)
```

"챗봇이 호출한 새 창인지" 구분값이 필요 없습니다

네이티브에서 `window.open`을 가로채 판별하려 하면 페이지의 다른 스크립트가 띄운 새 창과 구분할 수 없어 별도 식별값 설계가 필요합니다.
이 방식은 **전용 브릿지 이벤트로만 도착**하므로 출처가 챗봇임이 구조적으로 보장됩니다. `WKUIDelegate`나 `shouldOverrideUrlLoading`을 건드릴 필요도 없습니다.

3 웹 구현

```ts
// web/src/libs/linkHandler.ts
import { bridge } from './bridge';

// 자사 서비스로 간주할 호스트 — 실제 도메인으로 교체하세요.
const INTERNAL_HOSTS = ['m.example.com', 'www.example.com', 'example.com'];

function isInternalUrl(url: string): boolean {
  try {
    const host = new URL(url).hostname.toLowerCase();
    return INTERNAL_HOSTS.includes(host);
  } catch { return false; }
}

export function handleLinkClick({ url }: { url: string }): void {
  if (!url) return;

  // SDK 기본 동작과 맞춥니다 — 프로토콜이 없으면 https 를 붙입니다.
  const normalized = /^https?:\/\//i.test(url) ? url : `https://${url}`;

  if (!bridge.isNative()) {
    // 일반 브라우저에서는 SDK 기본 동작 유지
    window.open(normalized, '_blank', 'noopener,noreferrer');
    return;
```

```
  }

  if (isInternalUrl(normalized)) {
    bridge.send('navigate_internal', { url: normalized });  // 히스토리 유지
    return;
  }

  bridge.send('open_external', { url: normalized });        // 외부 뷰어
}
```

메신저에 연결

```
import { handleLinkClick } from '../libs/linkHandler';

<FixedMessenger
  appId={...}
  aiAgentId={...}
  handlers={{ onClickLink: handleLinkClick }}
>
```

호스트는 반드시 정확 일치로 검사하세요

`url.includes('example.com')` 같은 부분 문자열 비교는 `https://example.com.attacker.net/...` 을 자사 링크로 오인합니다. `new URL(url).hostname` 으로 꺼내 **목록과 정확히 일치**하는지 확인해야 합니다.

적용 범위

`onClickLink` 는 **마크다운 링크, 관리자 메시지 URL, 인용 링크, CTA 버튼, 이미지가 아닌 첨부파일 미리보기 클릭**에 모두 적용됩니다. 이미지·동영상의 내부 뷰어와 뷰어 안의 다운로드 버튼은 영향을 받지 않습니다.

4 자사 서비스 링크 — 히스토리를 남기는 두 가지 방식

방식	히스토리	메신저 상태
A. 별도 화면을 스택에 push (권장)	앱 내비게이션 스택이 유지	**보존** — 돌아오면 대화 내용과 스크롤 위치 그대로
B. 같은 WebView에서 이동	WebView 히스토리가 유지	메신저가 **리로드되어 초기화**

A를 권장합니다. B는 뒤로가기 자체는 동작하지만, 돌아왔을 때 메신저가 처음부터 다시 로드되어 대화가 초기 상태로 돌아갑니다.

Android

```
// 브릿지 이벤트 처리
"navigate_internal" -> {
    val url = data?.optString("url").orEmpty()
```

```
    mainHandler.post { activity.openInternalPage(url) }
}

private fun openInternalPage(url: String) {
    val scheme = runCatching { Uri.parse(url).scheme }.getOrNull()?.lowercase()
    if (scheme != "http" && scheme != "https") return

    // 새 Activity 가 스택에 쌓이므로 뒤로가기 = 메신저로 복귀
    startActivity(
        Intent(this, InternalWebActivity::class.java)
            .putExtra(InternalWebActivity.EXTRA_URL, url)
    )
}
```

B 방식을 택한다면, 메신저를 띄운 Activity 에 아래 처리가 있어야 합니다. (샘플에 기본 포함되어 있습니다)

```
override fun onBackPressed() {
    if (webView.canGoBack()) webView.goBack() else super.onBackPressed()
}
```

iOS

```
// 브릿지 이벤트 처리
case "navigate_internal":
    // ... http/https 스킴 검증 후 ...
    DispatchQueue.main.async { self.pushInternalPage(url) }


private func pushInternalPage(_ url: URL) {
    guard let top = Self.topViewController() else { return }
    let viewer = InternalWebViewController(url: url)
    // UINavigationController 로 감싸 뒤로·닫기 버튼을 제공
    let nav = UINavigationController(rootViewController: viewer)
    nav.modalPresentationStyle = .fullScreen
    top.present(nav, animated: true)
}
```

`InternalWebViewController`는 자체 `WKWebView`를 들고 `allowsBackForwardNavigationGestures = true`로 가장자리 스와이프 뒤로가기를 지원합니다. 메신저 WebView 는 그대로 살아 있으므로 닫으면 대화가 유지됩니다.

5 외부 링크 · PDF — 외부 뷰어 연결

iOS

`SFSafariViewController`를 사용합니다. **완료(닫기) 버튼과 공유 기능이 기본 제공**되고 PDF 렌더링도 OS가 담당합니다.

```
import SafariServices
```

```
case "open_external":
    // ... http/https 스킴 검증 후 ...
    DispatchQueue.main.async { self.presentExternalViewer(url) }

private func presentExternalViewer(_ url: URL) {
    guard let top = Self.topViewController() else { return }
    let viewer = SFSafariViewController(url: url)
    viewer.dismissButtonStyle = .close
    top.present(viewer, animated: true)
}
```

Android

`ACTION_VIEW`로 넘기면 OS가 기본 브라우저·PDF 뷰어를 띄우므로 **뒤로가기와 닫기가 기본 제공**됩니다.

```
"open_external" -> {
    val url = data?.optString("url").orEmpty()
    mainHandler.post { activity.openExternalViewer(url) }
}

private fun openExternalViewer(url: String) {
    val uri = runCatching { Uri.parse(url) }.getOrNull()
    val scheme = uri?.scheme?.lowercase()
    if (uri == null || (scheme != "http" && scheme != "https")) return

    val intent = Intent(Intent.ACTION_VIEW, uri).apply {
        addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    }
    try { startActivity(intent) }
    catch (e: ActivityNotFoundException) {
        Toast.makeText(this, "파일을 열 수 있는 앱이 없습니다.", Toast.LENGTH_SHORT).show()
    }
}
```

URL 스킴 검증은 반드시 넣으세요

`http` / `https` 만 허용합니다. 검증 없이 넘기면 의도하지 않은 스킴(`file://` , 커스텀 스킴 등)으로 앱 내부 화면이 열릴 수 있습니다. 내부 이동·외부 뷰어 **양쪽 모두** 해당됩니다.

6 이미지 뷰어의 다운로드 버튼

> 이미지 첨부를 눌러 뷰어가 열리면 우측 상단에 **다운로드 버튼**이 있습니다. 웹 브라우저에서는 동작하지만 **앱(웹뷰)에서는 눌러도 아무 일이 일어나지 않습니다.**

왜 동작하지 않는가

이 버튼은 `onClickLink`를 거치지 않는 **순수 HTML 앵커**입니다.

```
<a href="https://file-xxx.../image.jpg" download="image.jpg">
```

두 가지가 겹쳐서 막힙니다.

원인	설명
`download` 속성 무시	HTML `download` 속성은 **동일 오리진에서만** 동작합니다. 파일 URL은 샌드버드 CDN(다른 오리진)이라 브라우저가 속성을 무시하고 **일반 이동으로 처리**합니다.
웹뷰에 다운로드 기능 없음	그 이동마저 웹뷰가 받아주지 못합니다. Android는 `DownloadListener`, iOS는 `WKDownloadDelegate`를 앱이 구현해야 하며, **없으면 조용히 무시**됩니다.

SDK가 의도적으로 제외한 지점입니다

SDK 공식 문서는 이 버튼을 `onClickLink` 적용 대상에서 **의도적으로 제외**했다고 명시합니다 — 브라우저의 기본 다운로드 동작을 유지하기 위해서입니다. 따라서 앞 장의 링크 라우팅으로는 해결되지 않고, **네이티브 다운로드 처리를 따로 붙여야** 합니다.

Android 구현

```
// WebView 설정에 DownloadListener 를 붙입니다.
webView.setDownloadListener { url, userAgent, contentDisposition, mimeType, _ ->
    startFileDownload(url, userAgent, contentDisposition, mimeType)
}

private fun startFileDownload(
    url: String, userAgent: String?, contentDisposition: String?, mimeType: String?
) {
    val uri = runCatching { Uri.parse(url) }.getOrNull()
    val scheme = uri?.scheme?.lowercase()
    if (uri == null || (scheme != "http" && scheme != "https")) return

    // 서버가 준 Content-Disposition 과 MIME 으로 파일명을 추론
    val fileName = URLUtil.guessFileName(url, contentDisposition, mimeType)

    val request = DownloadManager.Request(uri).apply {
        setMimeType(mimeType)
        userAgent?.let { addRequestHeader("User-Agent", it) }
        setTitle(fileName)
        setNotificationVisibility(DownloadManager.Request.VISIBILITY_VISIBLE_NOTIFY_COMPLETED)
        setDestinationInExternalPublicDir(Environment.DIRECTORY_DOWNLOADS, fileName)
    }
    (getSystemService(DOWNLOAD_SERVICE) as DownloadManager).enqueue(request)
}
```

`DownloadManager`에 넘기면 알림·재시도·진행률을 OS가 모두 처리합니다. **Android 10(API 29)부터는 저장 권한이 필요 없습니다.** API 28 이하를 지원한다면 매니페스트에 다음을 추가합니다.

```
<uses-permission
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="28" />
```

iOS 구현

```
// 표시할 수 없는 응답(첨부 파일 등)은 다운로드로 전환
func webView(_ webView: WKWebView,
             decidePolicyFor navigationResponse: WKNavigationResponse,
             decisionHandler: @escaping (WKNavigationResponsePolicy) -> Void) {
    decisionHandler(navigationResponse.canShowMIMEType ? .allow : .download)
}

func webView(_ webView: WKWebView, navigationResponse: WKNavigationResponse,
             didBecome download: WKDownload) {
    download.delegate = self
}

// MARK: WKDownloadDelegate

func download(_ download: WKDownload, decideDestinationUsing response: URLResponse,
              suggestedFilename: String, completionHandler: @escaping (URL?) -> Void) {
    let dir = FileManager.default.temporaryDirectory.appendingPathComponent("downloads",
isDirectory: true)
    try? FileManager.default.createDirectory(at: dir, withIntermediateDirectories: true)
    let dest = dir.appendingPathComponent(suggestedFilename.isEmpty ? "file" :
suggestedFilename)
    try? FileManager.default.removeItem(at: dest)
    downloadedFileURL = dest
    completionHandler(dest)
}

// 완료되면 공유 시트로 '사진에 저장' / '파일에 저장' 을 제공
func downloadDidFinish(_ download: WKDownload) {
    guard let url = downloadedFileURL else { return }
    downloadedFileURL = nil
    DispatchQueue.main.async {
        guard let top = Self.topViewController() else { return }
        let sheet = UIActivityViewController(activityItems: [url], applicationActivities: nil)
        // iPad 는 popover 앵커가 없으면 크래시합니다.
        sheet.popoverPresentationController?.sourceView = top.view
        top.present(sheet, animated: true)
    }
}
```

iOS 버전

`WKDownloadDelegate` 는 **iOS 14.5 이상**에서 사용할 수 있습니다. 더 낮은 버전을 지원한다면 `@available` 로 분기하거나 아래의 버튼 비활성화 방식을 쓰세요.

7 다운로드 버튼 비활성화

다운로드를 지원하지 않기로 했다면, **동작하지 않는 버튼을 노출하는 것보다 숨기는 편이 낫습니다.** SDK 설정 한 줄로 가능합니다.

```
<FixedMessenger
  appId={...}
  aiAgentId={...}
  config={{
    conversation: {
      fileViewer: { downloadEnabled: false }   // 기본값: true
    },
  }}
>
```

이미지 뷰어 우측 상단의 다운로드 버튼만 사라지고, **뷰어의 나머지 기능(확대, 이전/다음, 닫기)은 그대로** 동작합니다.

선택 기준

다운로드가 필요하다 → 6장의 네이티브 구현
필요 없다 / 당장 배포가 급하다 → 7장의 버튼 숨김 (코드 한 줄, 네이티브 수정 불필요)

8 동작 확인

확인 항목	기대 결과
자사 서비스 링크 클릭	앱 안에서 이동하고, **뒤로가기 한 번으로 대화 복귀**
복귀 후 대화 상태	대화 내용과 스크롤 위치가 **그대로 유지**
PDF 첨부 클릭	외부 뷰어가 뜨고 **완료/뒤로 버튼**으로 복귀
이미지 첨부 클릭	기존과 동일하게 SDK 내부 뷰어 (변화 없음)
외부 도메인 링크 클릭	외부 뷰어로 열림
일반 브라우저에서 열기	`window.open` 기본 동작 유지 (새 탭)
이미지 뷰어 다운로드 버튼	Android는 알림과 함께 저장, iOS는 공유 시트 표시 (비활성화했다면 **버튼이 보이지 않아야** 함)

웹과 네이티브를 모두 적용해야 합니다

웹의 `onClickLink` 연결과 네이티브의 `navigate_internal` · `open_external` 처리가 **둘 다 있어야** 완성됩니다. 한쪽만 적용하면 링크를 눌러도 아무 일이 일어나지 않습니다.

참고 문서

SDK 공식 가이드 — How to handle link clicks in conversation messages

`sendbird/ai-agent-js` → `.discussions/conversation/handle-link-taps/react.md`

네이티브 SDK를 직접 쓰는 경우 같은 디렉터리의 `ios.md` / `android.md`에 해당 플랫폼 구현이 정리되어 있습니다.